Computer Organization 2026 Programming Assessment I

Computer Organization 2026 Programming Assessment I

> Announce date: March 19, 2026
> Exam date: March 26 & 27, 2026

0. Overview

This assignment aims to get familiar with the hardware/software interface: instruction set architecture (ISA), as well as the development environment and tools (e.g., compiler and RISC-V simulator) for RISC-V programming. In particular, this programming assignment is to write inline assembly code and run the developed code on a RISC-V ISA Simulator, Spike, to obtain the simulation results.

The assignments of this course are based on the RISC-V Spike simulator running on Ubuntu Linux 24.04.1. As our assignments are built upon open source projects, there will be unexpected compatibility issues if you choose a different development environment.

This assignment consists of three problems, with a total score of **100 points**. You will be given some test cases, and you must ensure that your code passes all these test cases, referred to as *public testcases*. Please note that there will be *hidden testcases* as well. Try to make your program handle special inputs properly. For details on how points are assigned, see Scoring Criteria.

Suggested Workflow

1. Read this document carefully to understand the assignment requirements.
2. Complete the inline assembly code for Problems 1, 2, and 3.
3. Test your code using local-judge to check all public testcases. Think about possible hidden testcases and consider common edge cases. Manually test any additional cases you think are important. For more details, see Test by Judge Program.
4. On the examination date in the computer classroom, you should follow the instructions in Submission of Your Assignment to submit your work after completing Problems 1, 2, and 3.

Prerequisite: Environment Setup

You have to follow the procedures listed in the **HW0** document to set up your development environment.

> - We will use **Docker** for the first computer-based exam on 3/26 & 3/27. Make sure you're familiar with basic Docker operations.
> - The computers in the exam environment will use Ubuntu. We will provide VSCode for students to use, the container will be pre-loaded, and no additional extensions will be installed.

Prerequisite: Docker basic instructions

- Check container status (In host system)

```
$ docker ps -a
```

- Start container (In host system)

```
$ docker start $CONTAINER_NAME
```

- Execute container (In host system)

```
$ docker exec -it $CONTAINER_NAME bash
```

- Exit container (In container)

```
$ exit
```

- Stop container (In host system)

```
$ docker stop $CONTAINER_NAME
```

1. Problems

In this assessment, you will complete three problems by implementing **RV64I + F Extension** inline assembly in a C program. The goal is to practice low-level programming and understand how assembly interacts with C.

- **Problem 1 (Insertion Sort, 30%)**: Implement the **insertion sort** algorithm using inline assembly to sort an array of floating-points.
- **Problem 2 (Binary Search, 30%)**: Implement the **binary search** algorithm using inline assembly to find the index of a target value in a sorted array.
- **Problem 3 (Linked-List Cycle, 40%)**: Implement the given algorithm for a linked-list using inline assembly. You will need to find the `meet` position and check if linked-list has a cycle.

Important Notes

- You must write your code inside `asm volatile()`. Any modifications outside of `asm volatile()` are not allowed.
- To quickly locate the position of `asm volatile()` sections, use Ctrl+F to search for `TODO` markers.
- The following C code contains only essential parts for the explanation of this assessment. Please use the full version downloaded from NCKU Moodle as the primary reference.
- The following are the details of both public and hidden testcases.

I. Problem 1. Insertion Sort (30%)

Insertion sort is a stable, in-place, and $O(N^2)$ time complexity sorting algorithm that builds a final sorted list one element at a time. It operates by picking an element and inserting it into its correct position within the already-sorted portion of the array. [1]

Pseudo code:

```
---------------------------------------------
Algorithm 1 Insertion Sort with Shift Count
---------------------------------------------
procedure INSERTION-SORT(A)
    n ← length(A)
    shift_cnt ← 0
    for i ← 1 to n - 1 do
        key ← A[i]
        j ← i - 1
        while j ≥ 0 and A[j] > key do
            A[j + 1] ← A[j]
            j ← j - 1
            shift_cnt ← shift_cnt + 1
        end while
        A[j + 1] ← key
    end for
    return shift_cnt;
end procedure
```

Source code:

```c
#include <stdio.h>
#include <string.h>

/* TODO: Implement your code below */
int insertionSort(float *p_a, int arr_size)
{
    int shift_cnt = 0;  /* shift count */

    for (int i = 1; i < arr_size; i++) {
        float *p_i = p_a + i;  /* index of i-th element */
        int j = i - 1;
```

```c
        asm volatile(/*Assembly code*/);
    }

    return shift_cnt;
}

int main(int argc, char *argv[])
{
    FILE *input = stdin;
    FILE *output_file = NULL;
    float target = -1;

    if (argc >= 2) {
        input = fopen(argv[1], "r");
        if (!input) {
            fprintf(stderr, "Error opening file: %s\n", argv[1]);
            return 1;
        }
    }

    // Read 'target' from the input file of problem 2 (2_*.txt) first
    if (argc >= 3) {
        output_file = fopen(argv[2], "r");
        if (output_file) {
            fscanf(output_file, "%f", &target);
            fclose(output_file);
        }
    }

    int arr_size;
    fscanf(input, "%d", &arr_size);
    float arr[arr_size];

    // Read 'floats' from input into the array
    for (int i = 0; i < arr_size; i++) {
        float data;
        fscanf(input, "%f", &data);
        arr[i] = data;
    }

    if (argc >= 2) {
        fclose(input);
    }

    float *p_a = &arr[0];

    // Call insertion sort function
    int shift_cnt = insertionSort(p_a, arr_size);

    p_a = &arr[0];

    // Write to output file
    if (argc >= 3 && argv[2]) {
        output_file = fopen(argv[2], "w");
        if (output_file) {
            // target
            fprintf(output_file, "%.1f\n", target);
            // arr_size
```

```
            fprintf(output_file, "%d\n", arr_size);
            // sorted array
            for (int i = 0; i < arr_size; i++) {
                fprintf(output_file, "%.1f ", arr[i]);
            }
            fprintf(output_file, "\n");
            fclose(output_file);
        }
    }

    // Print the result
    for (int i = 0; i < arr_size; i++) {
        printf("%.1f ", *p_a++);
    }
    printf("\n");
    printf("%d\n", shift_cnt);
    return 0;
}
```

The following is an example of a testcase.

Input:

```
10
9.0 5.0 8.0 3.0 4.0 10.0 1.0 6.0 2.0 7.0
```

Expected output:

```
1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0
27
```

Input Format:

- The first line contains an integer `N`, the size of the array is within the range: 1 ≤ `N` ≤ 1000.
- The second line contains `N` single-precision floating-point key values with one decimal place, i.e., 1.0 ≤ `key[N]` ≤ 1000.0.

Expected Output Format:

- The first line contains sorted array.
- The second line contains an integer, `shift count`, which is the times of elements shifted.

Program Execution Steps:

- Read the input: The first line (`10`) is the array size, and the second line (`9.0 5.0 8.0 3.0 4.0 10.0 1.0 6.0 2.0 7.0`) is the unsorted array.
- Execute the insertion sort algorithm using inline assembly to sort the array.

- Print the sorted array: `1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0`.
- Print the shift_cnt: `27` and end the program.

II. Problem 2. Binary Search (30%)

Binary search is an efficient, $O(logN)$ time complexity algorithm used to find a target value within a sorted array by repeatedly dividing the search interval in half. It compares the target to the midpoint, eliminating the half that cannot contain the value, and repeats until the target is found or the range is empty. [2]
Note that the inputs to Problem 2 are the outputs of Problem 1, as will be detailed in the following content.

Pseudo code:

```
---------------------------------------------
Algorithm 2 Binary Search
---------------------------------------------
procedure BINARY-SEARCH(A,target)
    left ← 0
    right ← length(A) - 1
    result ← -1
    while left ≤ right do
        mid ← ⌊(left + right)/2⌋
        if A[mid] = target then
            result ← mid
            break
        else if A[mid] < target then
            left ← mid + 1
        else
            right ← mid - 1
        end if
    end while
    return result
end procedure
```

Source code:

```c
#include <stdio.h>

/* TODO: Implement your code below */
int binarySearch(float *p_a, int arr_size, float target)
{
    int result = -1;

    asm volatile(/*Assembly code*/);

    return result;
}

int main(int argc, char *argv[])
{
    FILE *input = stdin;
```

```
    if (argc >= 2) {
        input = fopen(argv[1], "r");
        if (!input) {
            fprintf(stderr, "Error opening file: %s\n", argv[1]);
            return 1;
        }
    }

    // Read 'target'
    float target;
    fscanf(input, "%f", &target);

    // Read 'arr_size'
    int arr_size;
    fscanf(input, "%d", &arr_size);
    float arr[arr_size];

    // Read 'floats' from input into the array
    for (int i = 0; i < arr_size; i++) {
        float data;
        fscanf(input, "%f", &data);
        arr[i] = data;
    }

    if (argc >= 2) {
        fclose(input);
    }

    float *p_a = &arr[0];

    int index = binarySearch(p_a, arr_size, target);

    // Print the result
    printf("%d ", index);
    printf("\n");

    return 0;
}
```

The following is an example of a testcase.

Input:

```
8.0
?
?
```

Expected output:

```
7
```

Input Format:

- The first line contains a single-precision floating-point with one decimal place, `target`, to search for in the array (1.0 ≤ `target` ≤ 1000.0).
- The second line contains the `array size` filled by Problem 1. (This input is the output of Problem 1.)
- The third line contains the `sorted array` filled by Problem 1. (This input is the output of Problem 1.)

Expected Output Format:

- A single integer, within the range [1, 1000], representing the index of the `target` in the array, or `-1` if the `target` is not found.

Program Execution Steps:

- Read the input: The first line (`8.0`) is the target value, the second line and the third line are the `array size` and the `sorted array` which will be filled by Problem 1 after Problem 1 executed.
- Execute the binary search algorithm using inline assembly to find `8` in the sorted array.
- Print `7` as the index of `8` in the sorted array and end the program.

III. Problem 3. Linked-List Cycle (40%)

Linked-List Cycle is a classic problem that asks you to determine whether a singly linked-list contains a cycle. A cycle occurs when a node's next pointer points to a node that has already appeared earlier in the list instead of pointing to null. We use $O(N)$ time complexity algorithm called **Floyd Cycle Detection Algorithm** [3], also called Tortoise and Hare Algorithm, to solve this problem.

Pseudo code:

```
---------------------------------------------
Algorithm 3 Floyd Cycle Detection
---------------------------------------------
procedure HAS-CYCLE(head)
    (1) If linked-list is empty or 'head' node has no next node, then there is
no cycle.
    (2) Create two pointers, 'slow' and 'fast', both initialized to 'head'
node.
        Additionally, initialize a pointer 'meet' to NULL.
    (3) In each iteration, move the 'slow' pointer one step forward and the
'fast' pointer two steps forward.
    (4) If at any time after moving, 'slow' and 'fast' point to the same node,
then linked-list has a cycle.
        Record this node position to 'meet' and output the key value of 'meet'
pointer.
```

```
    (5) If 'fast' reaches NULL or 'fast->next' is NULL, then there is no
cycle.
```

Source code:

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

struct ListNode
{
    int val;
    struct ListNode *next;
};

/* TODO: Implement your code below */
bool hasCycle(struct ListNode *head)
{
    struct ListNode *slow = head;
    struct ListNode *fast = head;
    struct ListNode *meet = NULL;
    bool result = false;

    asm volatile(/*Assembly code*/);

    if (result)
        printf("%d\n", meet->val);
    else
        printf("NULL\n");
    return result;
}

int main(int argc, char *argv[])
{
    FILE *input = stdin;

    if (argc >= 2) {
        input = fopen(argv[1], "r");
        if (!input) {
            fprintf(stderr, "Error opening file: %s\n", argv[1]);
            return 1;
        }
    }

    int node_count, edge_count;

    // Read first line: node_count and edge_count
    if (fscanf(input, "%d %d", &node_count, &edge_count) != 2) {
        fprintf(stderr, "Error reading node and edge count\n");
        if (argc >= 2) fclose(input);
        return 1;
    }

    if (node_count == 0) {
        printf("NULL\n");
        printf("false\n");
        if (argc >= 2) fclose(input);
```

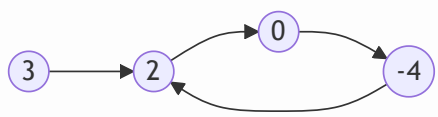
```
        return 0;
    }

    // Allocate nodes array
    struct ListNode **nodes = (struct ListNode**)malloc(node_count *
sizeof(struct ListNode*));

    // Read second line: node values and create nodes
    for (int i = 0; i < node_count; i++) {
        int val;
        if (fscanf(input, "%d", &val) != 1) {
            fprintf(stderr, "Error reading node value\n");
            if (argc >= 2) fclose(input);
            return 1;
        }
        nodes[i] = (struct ListNode*)malloc(sizeof(struct ListNode));
        nodes[i]->val = val;
        nodes[i]->next = NULL;
    }

    // Read edges and set next pointers
    for (int i = 0; i < edge_count; i++) {
        int from_val, to_val;
        if (fscanf(input, "%d %d", &from_val, &to_val) != 2) {
            fprintf(stderr, "Error reading edge\n");
            if (argc >= 2) fclose(input);
            return 1;
        }

        // Find 'from' and 'to' nodes by value
        struct ListNode *from_node = NULL;
        struct ListNode *to_node = NULL;

        for (int j = 0; j < node_count; j++) {
            if (nodes[j]->val == from_val) {
                from_node = nodes[j];
            }
            if (nodes[j]->val == to_val) {
                to_node = nodes[j];
            }
        }

        if (from_node && to_node) {
            from_node->next = to_node;
        }
    }

    if (argc >= 2) {
        fclose(input);
    }

    struct ListNode *head = (node_count > 0) ? nodes[0] : NULL;

    bool has_cycle = hasCycle(head);

    // Print the result
    if (has_cycle) {
        printf("true\n");
```

```
    } else {
        printf("false\n");
    }

    for (int i = 0; i < node_count; i++) {
        free(nodes[i]);
    }
    free(nodes);

    return 0;
}
```

The following is an example of a testcase.

Input:

```
4 4
3 2 0 -4
3 2
2 0
0 -4
-4 2
```



Expected output:

```
-4
true
```

Input Format:

- The first line contains two integers `N` and `E`, the number of nodes and the number of edges are defined as: 0 ≤ `N`, `E` ≤ 1000.
- The second line contains `N` key values of the `N` input nodes, -1000 ≤ `key[N]` ≤ 1000.
- Each of the next `E` lines contains two integers `from` and `to`, representing a directed edge from `from` to `to` (e.g., `1 2` means a directed edge from `1` to `2`), and the number of edges are always matches the number of nodes in the first line.
- Note
  - A node has at most two in-degree and one out-degree edges.
  - The directed graph must be weakly connected.

Expected Output Format:

- The first line contains an integer key value of `meet` pointer.
- The second line contains the result of having cycle or not. (e.g., Has cycle: `true`; no cycle: `false`)

Program Execution Steps:

- Read the input: The first line (`4 4`) are the number of nodes and the number of edges respectively, the second line (`3 2 0 -4`) are the key value of corresponding nodes, and after the third line (`3 2`) (`2 0`) (`0 -4`) (`-4 2`) represent the edges in the directed graph.
- Execute the given algorithm using inline assembly to find the position of `meet` pointer and show the directed graph has a cycle or not.
- Print the key value of `meet` pointer (`-4`) and existence of cycle (`true`), or `NULL` `false` if the directed graph doesn't has a cycle.

2. Implementation

I. PA1 Source file

We use local-judge to evaluate your program. For installation instructions, please refer to the **HW0** document. You can follow the steps below to generate a directory tree, depending on the environment you're using.

Docker Environment

If you used **Docker** for your environment setup, we provide source files `CO2026PA1.zip` that can be downloaded from the course website on NCKU Moodle. After unzipping the files into the mounted directory in host system, the directory tree of the source files in container is shown below.

```
.
├── bin
├── ...
└── workspace
    ├── CO_StudentID_PA1
    │   ├── 1_insertion_sort.c
    │   ├── 2_binary_search.c
    │   ├── 3_linked_list_cycle.c
    │   ├── Makefile
    │   ├── judge1.conf
    │   ├── judge2.conf
    │   └── judge3.conf
    ├── output
    ├── references
    │   ├── CO2026PA1.pdf
    │   ├── Computer_Organization_2026_Programming_Assessment_I.pdf
    │   ├── Computer_Organization_Homework_0.pdf
    │   ├── Extended_Asm_(Using_the_GNU_Compiler_Collection_(GCC)).pdf
```

```
    │   ├── GCC-Inline-Assembly-HOWTO.pdf
    │   └── riscv-unprivileged.pdf
    └── testcases
        ├── expected
        │   ├── 1_1.out
        │   ├── 1_2.out
        │   ├── 2_1.out
        │   ├── 2_2.out
        │   ├── 3_1.out
        │   └── 3_2.out
        └── input
            ├── 1_1.txt
            ├── 1_2.txt
            ├── 2_1.txt
            ├── 2_2.txt
            ├── 3_1.txt
            └── 3_2.txt
```

VirtualBox Environment

If you used **VirtualBox** for your environment setup, we provide source files `CO2026PA1.zip` that can be downloaded from the course website on NCKU Moodle. After unzipping the files into the VM directory, the directory tree of the source files in VirtualBox is shown below.

```
.
├── CO_StudentID_PA1
│   ├── 1_insertion_sort.c
│   ├── 2_binary_search.c
│   ├── 3_linked_list_cycle.c
│   ├── Makefile
│   ├── judge1.conf
│   ├── judge2.conf
│   └── judge3.conf
├── output
├── references
│   ├── CO2026PA1.pdf
│   ├── Computer_Organization_2026_Programming_Assessment_I.pdf
│   ├── Computer_Organization_Homework_0.pdf
│   ├── EExtended_Asm_(Using_the_GNU_Compiler_Collection_(GCC)).pdf
│   ├── GCC-Inline-Assembly-HOWTO.pdf
│   └── riscv-unprivileged.pdf
└── testcases
    ├── expected
    │   ├── 1_1.out
    │   ├── 1_2.out
    │   ├── 2_1.out
    │   ├── 2_2.out
    │   ├── 3_1.out
    │   └── 3_2.out
    └── input
        ├── 1_1.txt
        ├── 1_2.txt
        ├── 2_1.txt
        ├── 2_2.txt
```

```
        ├── 3_1.txt
        └── 3_2.txt
```

From Scratch Setup

If you built environment setup **from scratch**, we provide source files `CO2026PA1.zip` that can be downloaded from the course website on NCKU Moodle. After unzipping the files to your local directory, the directory tree of the source files is shown below.

```
.
├── CO_StudentID_PA1
│   ├── 1_insertion_sort.c
│   ├── 2_binary_search.c
│   ├── 3_linked_list_cycle.c
│   ├── Makefile
│   ├── judge1.conf
│   ├── judge2.conf
│   └── judge3.conf
├── output
├── references
│   ├── CO2026PA1.pdf
│   ├── Computer_Organization_2026_Programming_Assessment_I.pdf
│   ├── Computer_Organization_Homework_0.pdf
│   ├── Extended_Asm_(Using_the_GNU_Compiler_Collection_(GCC)).pdf
│   ├── GCC-Inline-Assembly-HOWTO.pdf
│   └── riscv-unprivileged.pdf
└── testcases
    ├── expected
    │   ├── 1_1.out
    │   ├── 1_2.out
    │   ├── 2_1.out
    │   ├── 2_2.out
    │   ├── 3_1.out
    │   └── 3_2.out
    └── input
        ├── 1_1.txt
        ├── 1_2.txt
        ├── 2_1.txt
        ├── 2_2.txt
        ├── 3_1.txt
        └── 3_2.txt
```

II. Internals of PA1 directory Structure

`CO_StudentID_PA1/`: This directory contains **source code and configuration files** for PA1.

- `1_insertion_sort.c`
  Implementation of **sorting an array**.
- `2_binary_search.c`
  Implementation of **search on an array**.

- `3_linked_list_cycle.c`
  Implementation of **detecting a cycle in linked-list**.
- `judge1.conf`, `judge2.conf`, and `judge3.conf` are configuration files for judging PA1 testcases: **insertion sort**, **binary search**, and **linked-list cycle**, respectively.

`output`: This folder is used to store temporary output files.

`references`: This folder contains the documents that you may refer to during the implementation.

`testcases/input/`: This folder contains **test input files**, the public testcases. Each file corresponds to a specific **testcase**.

- `1_1.txt`, `1_2.txt`
  Input files for **array sorting** (`1_insertion_sort.c`).
- `2_1.txt`, `2_2.txt`
  Input files for **array search** (`2_binary_search.c`).
- `3_1.txt`, `3_2.txt`
  Input files for **linked-list cycle detecting** (`3_linked_list_cycle.c`).

`testcases/expected/`: This folder contains **expected output files** for each testcase.

- `1_1.out`, `1_2.out`
  Expected outputs for **array sorting**.
- `2_1.out`, `2_2.out`
  Expected outputs for **array search**.
- `3_1.out`, `3_2.out`
  Expected outputs for **linked-list cycle detecting**.

III. Scoring Criteria

- Problem 1, 2, and 3 have a total of 100 points.
- Problem 1 has 2 public testcases, each worth **10** points, and 2 hidden testcases, each worth **5** points, for a total of 30 points.
- Problem 2 has 2 public testcases, each worth **10** points, and 2 hidden testcases, each worth **5** points, for a total of 30 points.
- Problem 3 has 2 public testcases, each worth **10** points, and 4 hidden testcases, each worth **5** points, for a total of 40 points.
- Public testcases account for **60** points, while hidden testcases account for **40** points.

IV. Test by Judge Program

Now, you can use the judge program to get the score of your developed code with the following commands.

```
$ cd /workspace/CO_StudentID_PA1
$ make judge
riscv64-unknown-linux-gnu-gcc -static -o 1 1_insertion_sort.c
riscv64-unknown-linux-gnu-gcc -static -o 2 2_binary_search.c
riscv64-unknown-linux-gnu-gcc -static -o 3 3_linked_list_cycle.c
judge -c judge1.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   1_1 | ✔
=======+==============================================================================
   1_2 | ✔
=======+==============================================================================
Correct/Total problems: 2/2
Obtained/Total scores:  20/20
or `judge -c judge1.conf -v 1`
judge -c judge2.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   2_1 | ✔
=======+==============================================================================
   2_2 | ✘
=======+==============================================================================
Correct/Total problems: 1/2
Obtained/Total scores:  10/20
or `judge -c judge2.conf -v 1`
judge -c judge3.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   3_1 | ✘
=======+==============================================================================
   3_2 | ✘
=======+==============================================================================
Correct/Total problems: 0/2
Obtained/Total scores:  0/20
or `judge -c judge3.conf -v 1`
```

If you want to judge a specific problem, you can use the command `judge -c [CONFIG]`. For example, you can check the result of problem 1 via the command `judge -c judge1.conf`.

```
$ cd /workspace/CO_StudentID_PA1
$ judge -c judge1.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
```

```
=======+==============================================================================
   1_1 | ✔
=======+==============================================================================
   1_2 | ✔
=======+==============================================================================
Correct/Total problems: 2/2
Obtained/Total scores:  20/20
```

If your code generates wrong output, you can use the parameter `-v 1` while running the judge command to check the differences between your output and the correct answer. The example message is shown as below.

```
Example:
2_1
standard answer is 7.
your problem output is 4.
2_2
standard answer is 34.
your problem output is -1.
```

```
$ cd /workspace/CO_StudentID_PA1
$ judge -c judge2.conf -v 1
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   2_1 | ✘
-------+------------------------------------------------------------------------------
--- /workspace/testcases/expected/2_1.out       2026-02-10 10:03:16.834147073 +0(
+++ ../output/2_1local_1771860089.out   2026-02-23 15:21:29.168058531 +0000
@@ -1 +1 @@
-7
+4

=======+==============================================================================
   2_2 | ✘
-------+------------------------------------------------------------------------------
--- /workspace/testcases/expected/2_2.out       2026-02-10 10:03:16.834147073 +0(
+++ ../output/2_2local_1771860089.out   2026-02-23 15:21:29.184058527 +0000
@@ -1 +1 @@
-34
+-1

=======+==============================================================================
Correct/Total problems: 0/2
Obtained/Total scores:  0/20
```

V. Manually Test Possible Hidden Testcases

If you want to test a different testcase, modify the files in testcases/expected and testcases/input according to the instructions for each problem.

3. Programming Assessment Guidelines

I. Guidelines

- We will announce programming assessment this week for you to practice at home. Next week, the exam will cover the same content.
- Due to the limited number of seats in the computer classroom, we offer two exam dates, March 26 and March 27, with the same time period, so you can choose the one you prefer.
- Please bring your student ID card as proof of identity.
- **During the computer-based exam:**
  - The network need to be completely disconnected.
  - The use of USB/external devices, cheat sheets, paper notes, or any AI tools is strictly prohibited.
  - **Violations of the above rules will be regarded as cheating, and the exam score will be recorded as 0.**
  - There will be no hints provided during the exam.
  - You may only access the NCKU Moodle while uploading your submission.

II. Submission of Your Assessment

- Submit your zip file to NCKU Moodle.
- The zip file organization of your directory should be as follows:

```
CO_F12345678_PA1.zip
└── CO_F12345678_PA1
    ├── 1_insertion_sort.txt
    ├── 2_binary_search.txt
    └── 3_linked_list_cycle.txt
```

- **NOTE**:
  - Change `CO_StudentID_PA1` to your student ID number (e.g., `CO_F12345678_PA1`).
  - Each `*.txt` file should contain only **ASSEMBLY CODE**.
    For example:

```
"add %[a], %[a], %[b]\n\t"
:[a] "+r"(a)
:[b] "r"(b)
```

- Do not submit any files or contents that are not listed above.

> - Do not modify the `Makefile`, as this may cause the judge program to fail, resulting in a score of zero.

> - Incorrect format (either the file structure or file name) will lose 10 points.

4. References

- GCC-Inline-Assembly-HOWTO
- Extended Asm - Assembler Instructions with C Expression Operands
- The RISC-V Instruction Set Manual Version 2.2
- Local Judge
- Docker